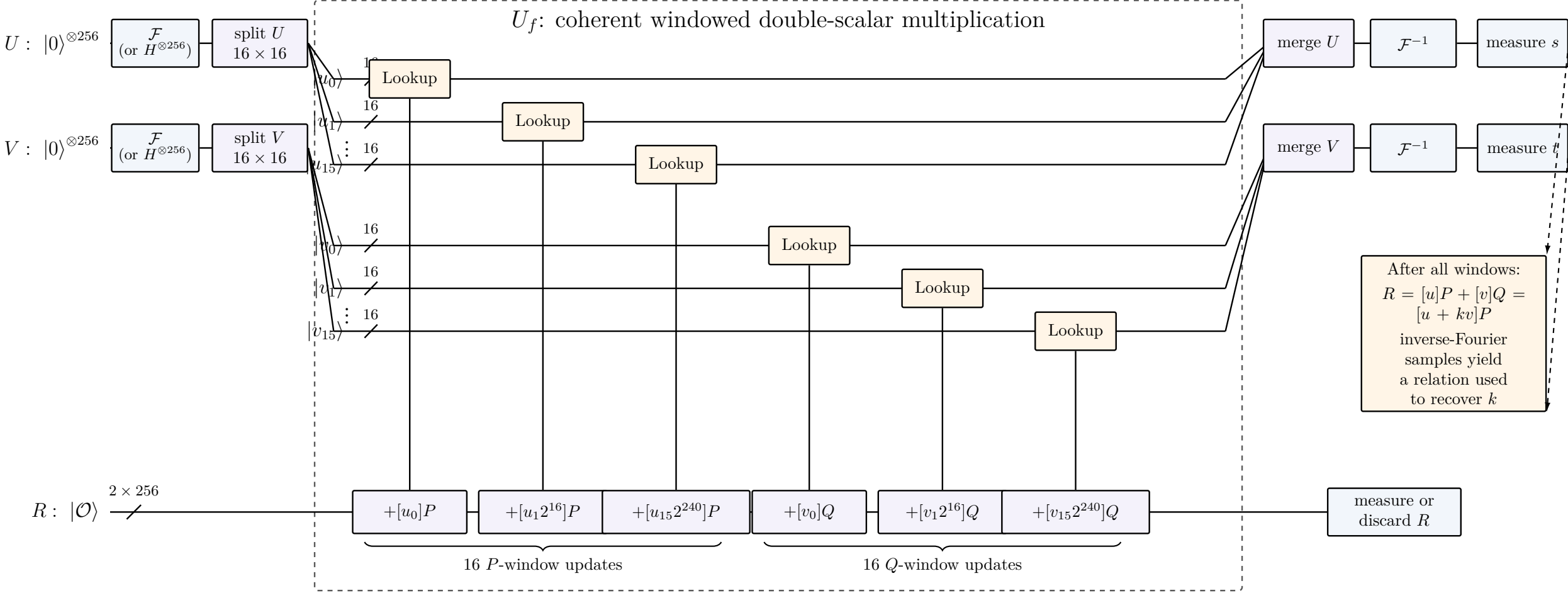


Shor's ECDLP circuit with the window controls expanded

Given P and $Q = [k]P$, evaluate $f(u, v) = [u]P + [v]Q = [u + kv]P$



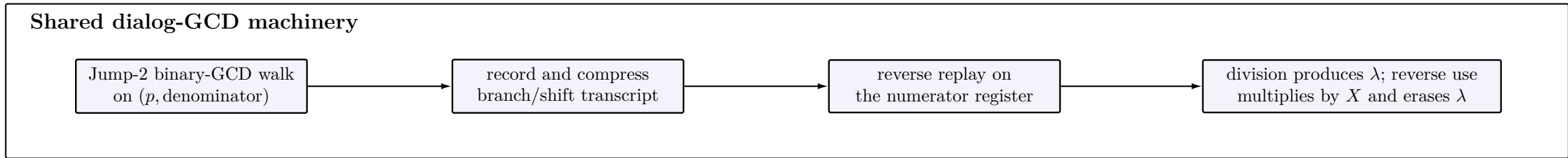
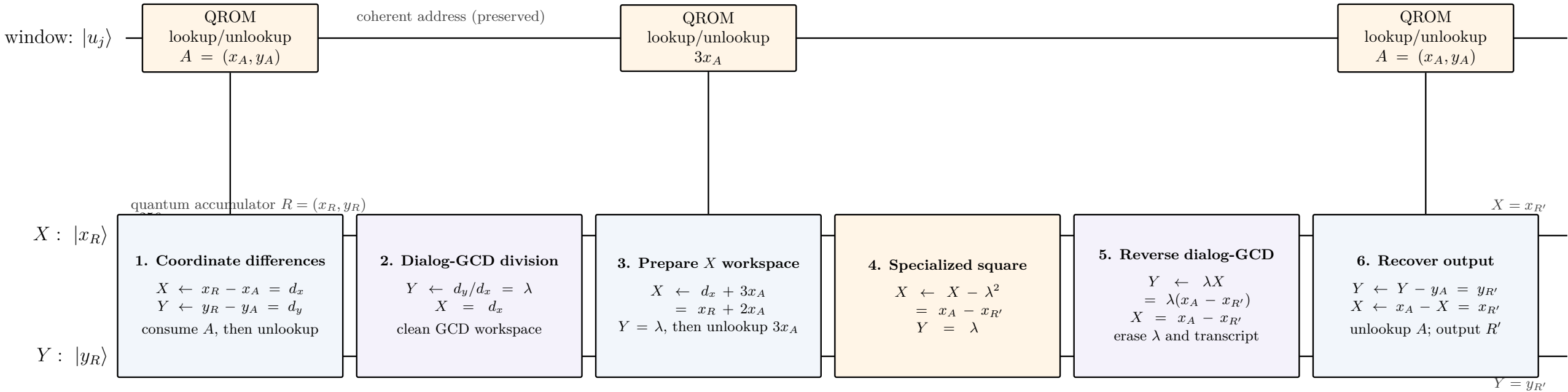
The window registers pass through U_f unchanged: they are coherent QROM addresses controlling additions on R , not individual CCX-style controls.

For $n = 256$ and $w = 16$ there are 32 conceptual window updates; boundary optimizations reduce this to 28 point-addition calls in the paper's `secp256k1` estimate.

Conceptual fully coherent view; the semiclassical implementation interleaves Fourier operations with the windows and recycles the control qubits.

Inside one window-controlled mixed point addition

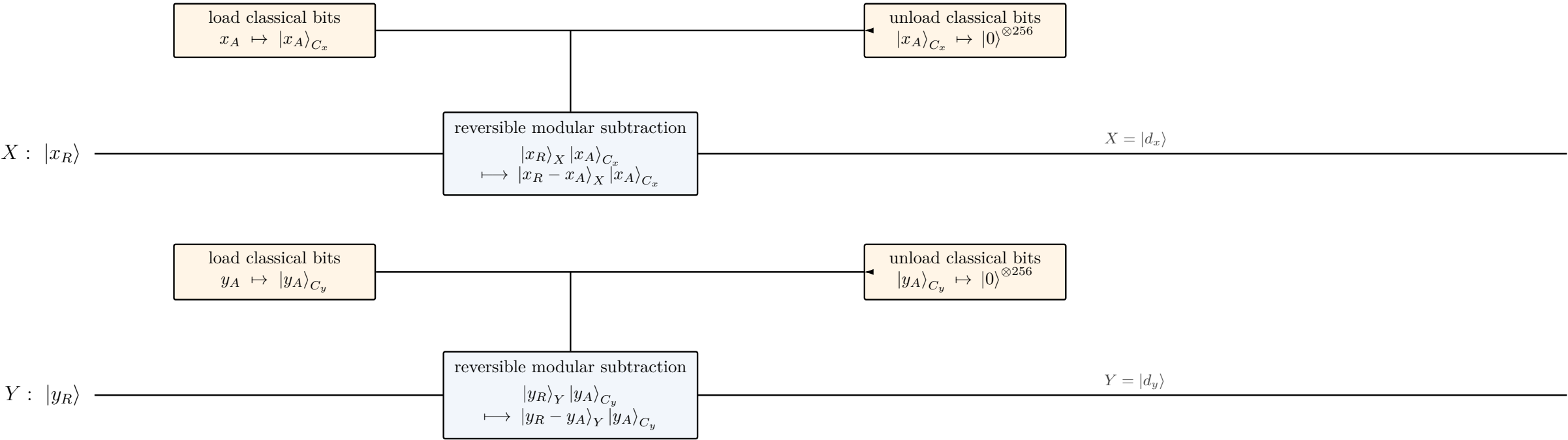
Example: $+[u_1 2^{16}]P$, with selected addend $A = [u_1 2^{16}]P$ and output $R' = R + A$



When $u_j = 0$, the table selects the identity $\mathcal{O}$ and the controlled arithmetic leaves R unchanged.
In ECDSA.Fail, (x_A, y_A) are classical input bits; a full windowed-Shor circuit must separately implement and validate the coherent QROM interface.

Phase 1: form the coordinate differences

$$d_x = x_R - x_A \pmod{p}, \quad d_y = y_R - y_A \pmod{p}$$



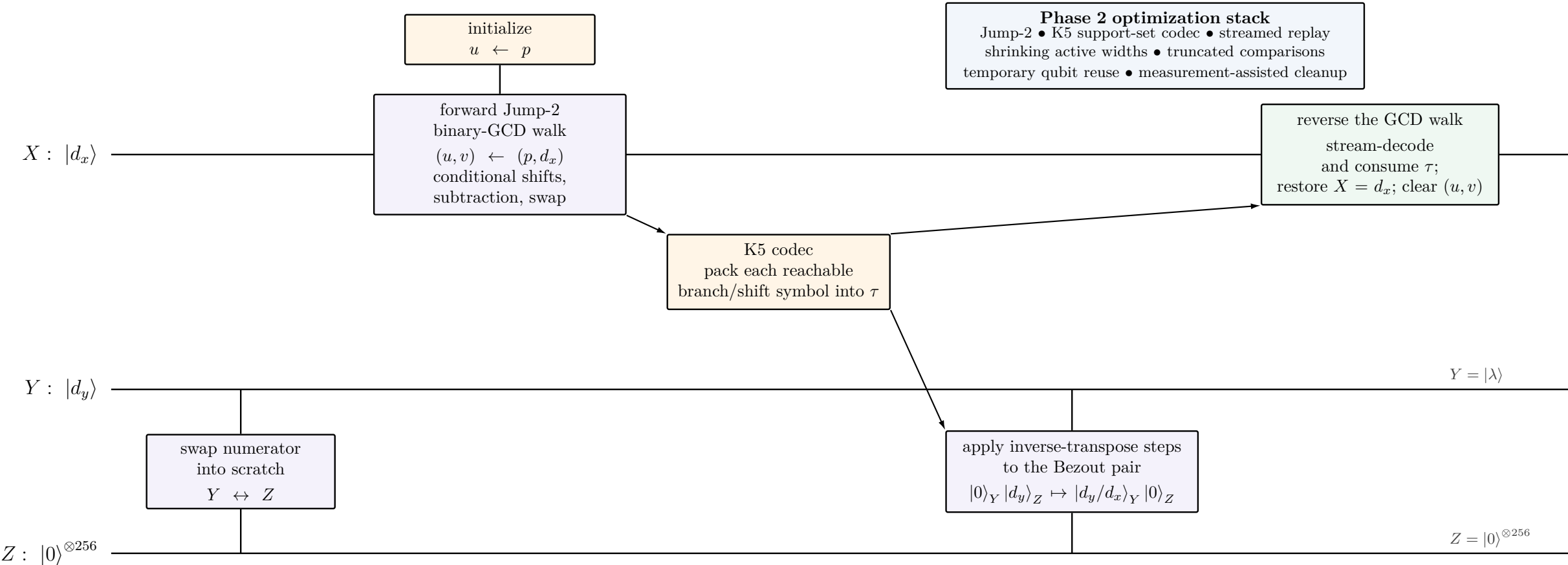
Reversible-state summary

$$|x_R\rangle_X |y_R\rangle_Y |0\rangle_{C_x} |0\rangle_{C_y} \mapsto |d_x\rangle_X |d_y\rangle_Y |0\rangle_{C_x} |0\rangle_{C_y}$$

The two coordinate subtractions are independent. The classically known addend is materialized only for the duration of each arithmetic block and is then removed, so no copy of (x_A, y_A) remains live.

Phase 2: compute the slope by dialog-GCD division

$\lambda = d_y d_x^{-1} \pmod{p}$ while restoring $X = d_x$ and clearing the transcript



Dialog relation and cleanup

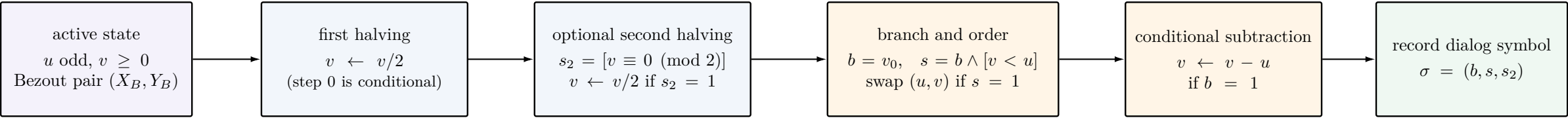
$$(u, v; X_B, Y_B) : (p, d_x; 0, d_y) \xrightarrow{\text{walk and inverse-transpose replay}} (1, 0; \lambda, 0), \quad \lambda = d_y d_x^{-1} \pmod{p}$$

Each Jump-2 step records whether subtraction, swapping, and a second halving occurred. Only five of the eight nominal three-bit symbols are reachable, which permits the compressed dialog representation.

Source correspondence: `mod_mul_inverse_in_place(..., Direction::Inverse)`, `forward_gcd_jump`, and `reverse_gcd_jump`.

Phase 2 detail: the Jump-2 Euclidean step

Up to two binary-GCD halvings are represented by one reversible control symbol



Only five symbols are reachable

$s_2 = 0 \Rightarrow b = 1$, because the value remaining after the first halving is odd;
 $b = 0 \Rightarrow s = 0$, because swapping is enabled only on a subtraction step.

(1, 0, 0)

(1, 1, 0)

(0, 0, 1)

(1, 0, 1)

(1, 1, 1)

Why the recorded symbol is sufficient

The walk applies a data-dependent linear update M to (u, v) . The paired arithmetic applies the inverse transpose M^{-T} to (X_B, Y_B) , preserving the division relation. The bit s_2 selects whether this paired update contains one or two powers of two, while b and s select subtraction and operand order.

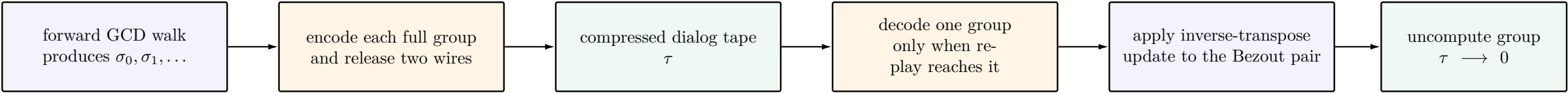
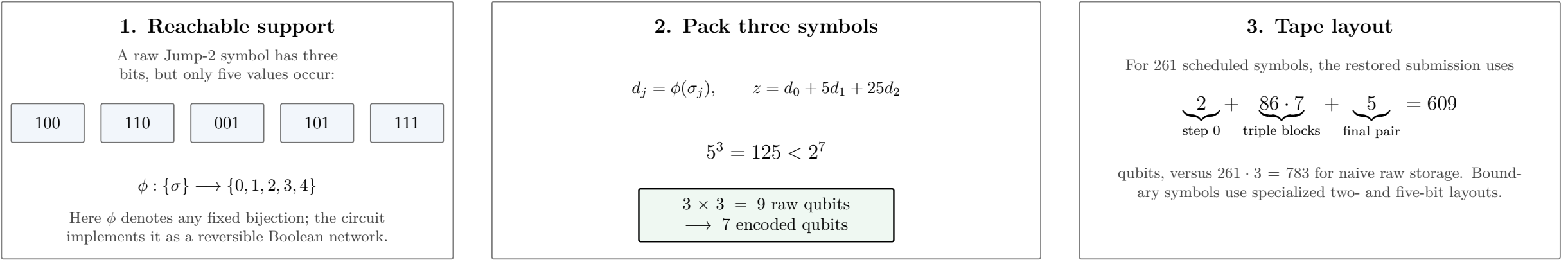
$$\text{walk: } (u, v) \xrightarrow{M} (u', v') \iff \text{paired update: } (X_B, Y_B) \xrightarrow{M^{-T}} (X'_B, Y'_B)$$

Compared with a one-step binary-GCD schedule, Jump-2 packages the mandatory halving and a possible second halving into one recorded iteration. Submission 8e9c9a2 executes 261 such iterations and replays their symbols in reverse order to recover the quotient while restoring the Euclidean workspace.

Implementation correspondence: *forward_gcd_jump*, *apply_step_reverse*, *apply_step_forward*, and *reverse_gcd_jump*.

Phase 2 detail: K5 codec and streamed dialog replay

Compress the five-way Jump-2 control alphabet, then decode only the symbols needed by the active replay step



The width reduction comes from scheduling as well as encoding



As the active Euclidean registers narrow, the tape grows into the released space. Streamed decoding prevents the raw transcript and the full arithmetic state from being live simultaneously.

Phase 2 detail: width scheduling and qubit reuse

The GCD state becomes smaller as the dialog grows, and short-lived workspaces are fitted into the released lanes

Scheduled active widths

$$w_i = \text{SCHED_J2}[i]$$

At iteration i , the implementation retains only the active w_i -bit slices of u and v . High-order lanes whose values are known to be zero are released, so shifts, swaps, and conditional subtraction become progressively narrower.

shrinking (u, v) creates
room for the growing tape τ

Truncated comparison windows

$$k_i = \text{cmp_window}(i, w_i) < w_i$$

The swap decision compares only a scheduled portion of the active operands. This reduces comparator work and scratch, but the selected windows are calibrated under the benchmark’s validation model rather than justified here as an all-input exact replacement.

smaller comparison window
 $\Rightarrow$ fewer gates and ancillas

Temporary hosting and cleanup

The known bits $u_0 = 1$ and $v_0 = 0$ can be parked while dormant, exposing their physical lanes to temporary state. Already-live registers also serve as dirty “vent” space for chunked adders, with every borrowed value restored before its owner becomes active again.

Measurement-assisted uncomputation clears selected AND and carry ancillas using classical phase feedback, shortening their ownership intervals without changing the logical map.

Combined live-width model

$$Q_i \approx \underbrace{512}_{\text{Bezout pair}} + \underbrace{|\tau_i|}_{\text{compressed dialog}} + \underbrace{2w_i}_{u_i, v_i} + \underbrace{q_{\text{work}, i}}_{\text{comparison, carry, and control}}$$

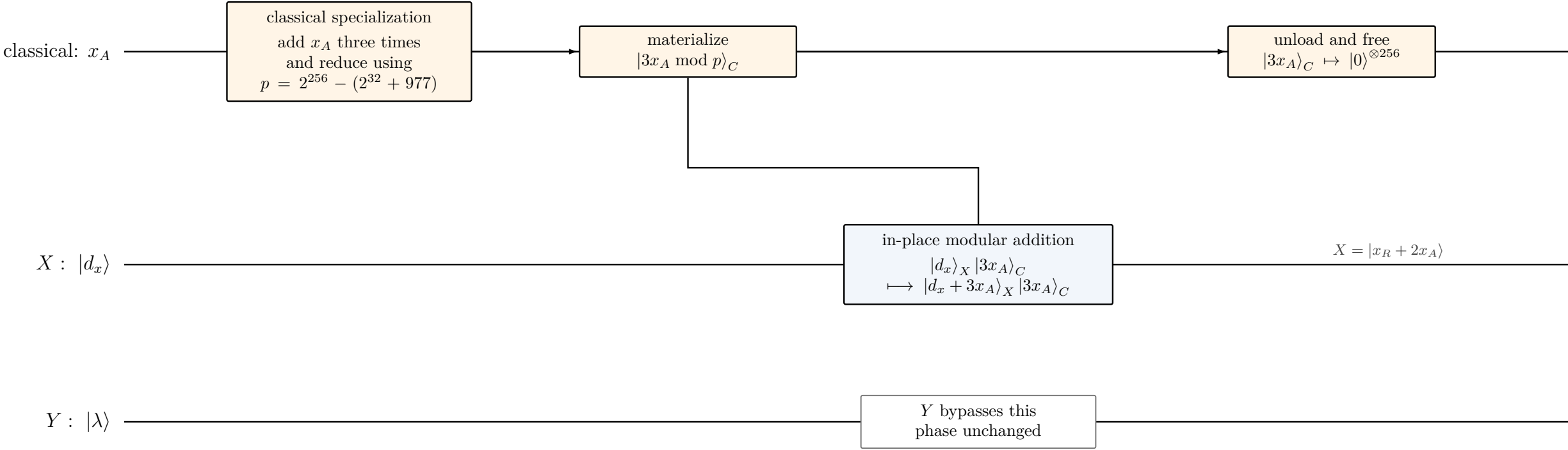
The central scheduling objective is to make the decrease in $2w_i$ absorb the growth of $|\tau_i|$, while hosting and measurement-based cleanup keep $q_{\text{work}, i}$ below the same peak.

Exact reversible mechanisms include support-set encoding, streamed replay, known-bit parking, restoration of borrowed dirty lanes, and phase-corrected measurement uncomputation. The chosen narrowing and comparison schedules carry the finite-support evidential status of the submitted benchmark circuit.

Implementation correspondence: SCHED_J2, cmp_window, park_known_one, park_known_zero, with_dirty_vent_pool, and clear_and.

Phase 3: prepare the X workspace

$X = d_x \mapsto d_x + 3x_A = x_R + 2x_A \pmod p, \quad Y = \lambda$



Why the transformation prepares the square step

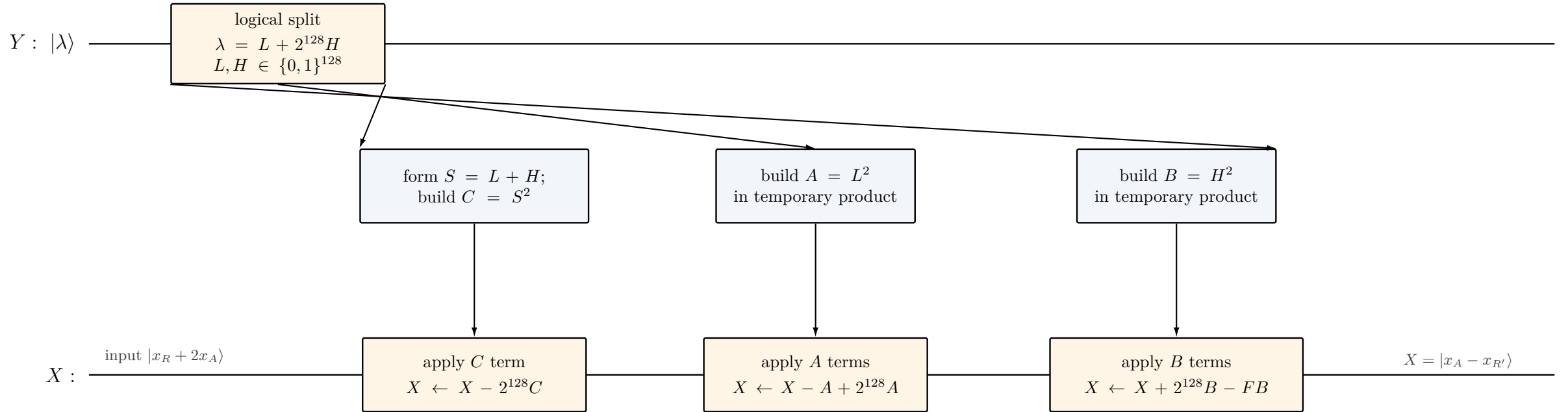
$$d_x + 3x_A = (x_R - x_A) + 3x_A = x_R + 2x_A$$

The following phase subtracts λ^2 , producing $x_R + 2x_A - \lambda^2 = x_A - x_{R'}$. The submitted route uses its configured low-cost modular-adder variant; the classical multiplier does not occupy quantum storage outside this phase.

Source correspondence: `tlm_coord_add3x`, `classical_times3_mod_q`, and `coord_add3x`.

Phase 4: subtract the specialized modular square

$$X \mapsto X - \lambda^2 = x_A - x_{R'} \pmod{p}, \quad p = 2^{256} - F, \quad F = 2^{32} + 977$$



Identity implemented by the streamed square

$$\lambda^2 = A + 2^{128}(C - A - B) + 2^{256}B \equiv A + 2^{128}(C - A - B) + FB \pmod{p}$$

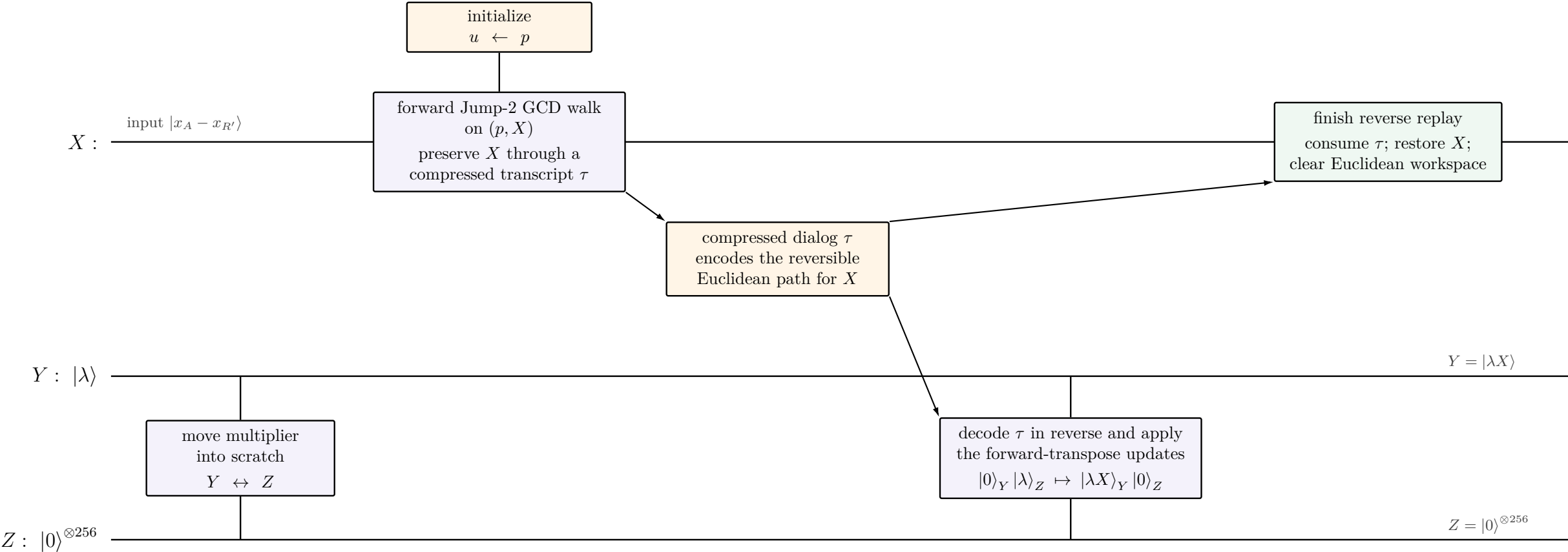
The signed updates shown above sum to $-\lambda^2$. Each of A , B , and C is built, accumulated into X , and reversibly unbuilt before the next product is retained; $Y = \lambda$ is never overwritten.

The reduction of the $2^{256}B$ term to FB exploits the pseudo-Mersenne form of the secp256k1 prime and avoids a generic full-width modular-square workspace.

Source correspondence: *mod_square_sub_pm_secp256k1_symmetric*, including the C , A , and B build/apply/unbuild phases.

Phase 5: multiply by replaying the dialog-GCD circuit in reverse

$$Y = \lambda \mapsto \lambda X = \lambda(x_A - x_{R'}) \pmod{p} \text{ while preserving } X$$



Why the division machinery is reused

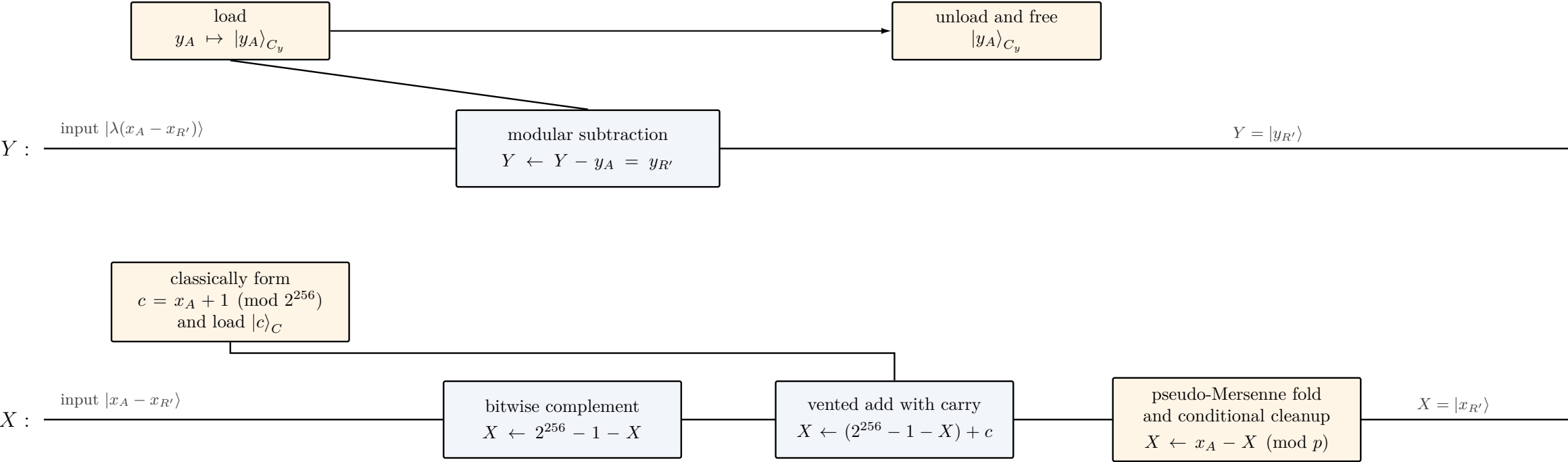
Multiplication by a quantum value must remain reversible. The source therefore runs the same in-place division construction in the opposite direction: the GCD path for X is recorded, replayed onto $(0, \lambda)$, and then erased.

$$|X\rangle_X |\lambda\rangle_Y |0\rangle_Z \mapsto |X\rangle_X |\lambda X\rangle_Y |0\rangle_Z$$

Source correspondence: `mod_mul_inverse_in_place(..., Direction::Forward)` and reverse replay with `apply_step_forward`.

Phase 6: recover the affine output coordinates

$$y_{R'} = \lambda(x_A - x_{R'}) - y_A, \quad x_{R'} = x_A - (x_A - x_{R'})$$



Final reversible map

$$|x_A - x_{R'}\rangle_X |\lambda(x_A - x_{R'})\rangle_Y \longmapsto |x_{R'}\rangle_X |y_{R'}\rangle_Y$$

The fused reverse subtraction uses $\overline{X} + (x_A + 1) = x_A - X \pmod{2^{256}}$ and then applies the secp256k1 modular correction.
All loaded classical coordinates, carry state, and temporary reduction data are removed before returning the two quantum output registers.

Source correspondence: `t1m_coord_y_sub_final`, `t1m_coord_rsub_final`, and the enabled `fused_mod_rsub_vented_loaded` route.